

ui-connector deploy runbook (header auth)

2026-09-22 · @Someone

Back up the running image, build the header-auth release with Cloud Build, pin it on `adt-ui-connectorv2-1` by digest, verify it, and keep a one-command rollback ready.

Before you start

This release reads the token **only** from `Authorization: Bearer <token>`. The LWC can't set headers on a browser WebSocket, which is why `?token=` was adopted in July, so LWC connections will close with `1008 missing_token` until the client changes. Test with Postman and keep step 6 ready.

- **Where:** `gcloud` on the bastion or Cloud Shell; `kubectl` on the bastion. Every command uses long flags.
- **Files:** unzip `ui-connector.zip` there. The build folder must hold `Dockerfile`, `main.py`, `utils.py`, `requirements.txt` and the `connector/` folder.
- **Dockerfile:** use the one from the zip. Your current one copies only `main.py`, which crash-loops on `ModuleNotFoundError`, the same failure as in August. The new one fails the build instead, via its `IMPORT OK` check.
- **TST:** same steps with `PROJ=adtgcp-ent-tst-ccs-tlcm-8143` and the TST cluster context, namespace and deployment names.

1. Set variables

Short names keep typing down. `IMG` is read from the live Deployment, so it is the repo you already push to. If the session drops, run this block again.

```
export PROJ=adtgcp-ent-dev-ccs-tlcm-fe10
export NS=ui-connector
export DEP=adt-ui-connectorv2-1
gcloud config set project $PROJ
kubectl config set-context --current --namespace=$NS

export CUR=$(kubectl get deployment/$DEP \
  --output=jsonpath='{.spec.template.spec.containers[0].image}')
export IMG=${CUR%[@:]*}
echo "running: $CUR"
echo "repo:    $IMG"
```

repo should look like `us-central1-docker.pkg.dev/adtgcp-ent-dev-ccs-tlcm-fe10/ui-connector/ui-connector`, with no tag or digest on the end.

2. Back up the current artifact

Three copies: the exact digest for rollback, a named tag in Artifact Registry so a cleanup policy or overwrite can't lose it, and the code plus live manifest on disk.

```
# exact bytes running now (always a digest)
export OLD=$(kubectl get pods --selector=app=$DEP \
  --output=jsonpath='{.items[0].status.containerStatuses[0].imageID}')
export OLD=${OLD#docker-pullable://}
echo $OLD | tee ~/rollback-image.txt

# pin it in Artifact Registry under a readable name
gcloud artifacts docker tags add $OLD $IMG:backup-pre-header-auth

# download the code and the live Deployment (env vars included)
export POD=$(kubectl get pods --selector=app=$DEP \
  --output=jsonpath='{.items[0].metadata.name}')
mkdir -p ~/backup-current
kubectl cp $POD:/app ~/backup-current/app
kubectl get deployment/$DEP --output=yaml > ~/backup-current/deployment.yaml
ls ~/backup-current/app
```

`ls` should show `main.py` and `requirements.txt`. `tar: Removing leading '/' from`
`kubectl cp` is harmless.

See what the repo holds: `gcloud artifacts docker images list $IMG --include-tags --sort-by=~CREATE_TIME --limit=5`

Optional, the whole image as one file (needs Docker, e.g. Cloud Shell):

```
gcloud auth configure-docker us-central1-docker.pkg.dev --quiet
docker pull $OLD
docker save --output ~/backup-current/image.tar $OLD
```

3. Build and push the new image

Cloud Build builds and pushes, so the machine you run it from needs no Docker. Use a new tag per release; never overwrite `v2`.

```
cd ~/ui-connector
ls connector          # must list: auth  routes  ws  app.py  config.py ...
export TAG=v3-header-auth
gcloud builds submit --tag $IMG:$TAG .
```

The build log must show `IMPORT OK`. If it doesn't, you're in the wrong folder or on the old Dockerfile.

```
export NEW=$(gcloud artifacts docker images describe $IMG:$TAG \
  --format='value(image_summary.fully_qualified_digest)')
echo $NEW
```

`NEW` ends in `@sha256:...`; that exact string is what gets deployed.

4. Deploy to GKE

Pin by digest, not tag. With a tag, a node's cached copy can keep serving old code, which was the `:v2` problem in August.

```
kubectl set image deployment/$DEP "*= $NEW"
kubectl rollout status deployment/$DEP --timeout=180s
kubectl get pods --selector=app=$DEP
```

- **Env vars:** no changes needed. `WS_TOKEN_AUDIENCE`, `ALLOWED_SAS`, `REDISHOST` and the rest keep their names. Check with `kubectl set env deployment/$DEP --list`.
- **Manifest:** `kubectl set image` changes the live Deployment only. Put `$NEW` on the `image:` line of your `deployment.yaml` too, or the next `kubectl apply` brings the old image back.

5. Verify

```
# pods run the new digest (compare with: echo $NEW)
export COLS='POD:.metadata.name,IMAGE:.status.containerStatuses[0].imageID'
kubectl get pods --selector=app=$DEP --output=custom-columns="$COLS"

# startup line: "- | ui-connector | Started"
kubectl logs deployment/$DEP --tail=30

# health endpoints
kubectl port-forward deployment/$DEP 8080:8080 &
curl -s localhost:8080/
curl -s localhost:8080/auth-health
kill %1
```

Expect "redis": "connected" from /, and "google_certs": "reachable" plus "audience_configured": true from /auth-health.

Postman, New → WebSocket:

Test	URL	Header	Expected
Header auth	wss://dev.agentassist.adt.com/ws?conversation_id=conv-test1234	Authorization: Bearer <fresh token>	HISTORY_BATCH frame
No header	same URL	none	close 1008 missing_token
Old style	same URL + &token=<token>	none	close 1008 missing_token

Then `kubectl logs deployment/$DEP --tail=50 | grep "conv-test1234 |"` should show the `Connected` and `Session closed` lines.

6. Roll back

This goes back to the exact image saved in step 2. It works from a fresh session because the digest is in a file; run step 1 first so `DEP` and the namespace are set.

```
kubectl set image deployment/$DEP "*= $(cat ~/rollback-image.txt)"
kubectl rollout status deployment/$DEP --timeout=180s
```

`kubectl rollout undo deployment/$DEP` does the same if nothing else changed on the Deployment since. The old code is also in `~/backup-current/app` and under the `backup-pre-header-auth` tag. If you updated `deployment.yaml` in step 4, put the old image line back there too.